

GELİŞMİŞ RAG TEKNİKLERİ: CÜMLE PENCERESİ ERİŞİMİ RAPORU

LlamaIndex ve TruLens ile Ayırıştırılmış Vektör Arama ve Bağlamsal Sentezleme Mimarisi

1. YÖNETİCİ ÖZETİ

Geleneksel RAG (Retrieval-Augmented Generation) sistemleri, bilgi arama ve yanıt sentezleme adımlarında aynı metin bloklarını (chunk) kullanır. Bu durum, çözülmesi gereken yapısal bir çelişki doğurur: Geri çağırma (retrieval) süreci anlamsal benzerlik aramaları için küçük ve spesifik metin parçalarında en iyi sonucu verirken; dil modelleri (LLM) zengin, mantıklı ve eksiksiz yanıtlar üretebilmek için geniş bir bağlam penceresine ihtiyaç duyar. Eğer metin parçaları küçük tutulursa arama doğruluğu artar ancak sentez kalitesi bağlam eksikliğinden ötürü düşer. Metin parçaları büyük tutulduğunda ise gürültü artar ve anlamsal arama performansına olumsuz yansır.

Cümle Penceresi Erişimi (Sentence-Window Retrieval), bu iki aşamayı birbirinden ayırarak (decoupling) çelişkiyi ortadan kaldırır. Bu mimaride dokümanlar, indeksleme aşamasında tekil cümleler halinde vektörleştirilirken, her cümlenin komşu cümleleri metadata alanında saklanır. Geri çağırma sırasında yalnızca en alakalı cümleler (küçük düğümler) hızlıca tespit edilir. Sentezleme adımında ise tespit edilen bu cümleler, dil modeline gönderilmeden önce metadata'dan çekilen geniş çevreleyen bağlam pencereleriyle otomatik olarak değiştirilir. Böylece arama yüksek doğrulukla dar bağlamda gerçekleştirilirken, yanıt sentezi geniş ve eksiksiz bir bağlam penceresinde yapılır.

2. GÜNLÜK HAYAT ANALOJİSİ: BÜYÜTEÇ VE KİTAP SAYFASI

Sentence-Window Retrieval yönteminin çalışma prensibini ve geleneksel RAG ile farkını anlamak için kütüphanede araştırma yapan bir insanı hayal edebiliriz:

Büyüteç ve Kitap Sayfası Analojisi

Geleneksel RAG Yaklaşımı: Aradığınız cevabı bulmak için bir kitaba bakarsınız. İlgili cümleyi bulduğunuzda makasla o tek cümleyi kesip yanınıza alırsınız. Bu cümle elinizdedir ancak konunun bütünüyle olan ilişkisini (öncesinde ne tartışıldığını veya sonrasında hangi örneğin verildiğini) bilmediğiniz için, cevabınız eksik veya yüzeysel kalır. Bağlam tamamen kopmuştur.

Sentence-Window Yaklaşımı: Kütüphanede elinizde güçlü bir **büyüteç** olduğunu hayal edin. Büyüteci sayfalar üzerinde gezdirerek tam olarak aradığınız anahtar cümleye odaklanırsınız (yüksek benzerlik ve hızlı arama). Ancak cümleyi okumaya başladığınızda, büyütecin odağını otomatik olarak genişletirsiniz. Sadece o cümleyi değil, cümlenin sağında, solunda ve üstünde yer alan tüm paragrafı (bağlam penceresini) birlikte okuyarak resmi bir bütün olarak görürsünüz. Sonuç olarak, hem aradığınız yeri nokta atışı bulmuş olursunuz hem de bağlamı kaybetmediğiniz için son derece tutarlı, zengin ve doğru bir açıklama yapabilirsiniz.

3. TEKNİK MİMARİ VE BİLEŞENLER

Cümle Penceresi Erişimi mimarisinin kurulması ve sorgu motorunun (query engine) devreye alınması için LlamaIndex kütüphanesinde üç ana bileşen kullanılır:

1. SentenceWindowNodeParser: Dokümanları paragraflara veya sabit karakter uzunluklarına göre değil, tekil cümle düzeyinde böler. Her düğüm (node) bir cümle içerir. Ancak parser, her cümle için önüne ve arkasına belirtilen pencere boyutunda (window_size) komşu cümleleri metadata (özel 'window' anahtarı altında) olarak ekler. Örneğin pencere boyutu 3 ise, metadata o cümle için öncesindeki 3 cümleyi ve sonrasındaki 3 cümleyi içerecektir.

2. MetadataReplacementPostProcessor: Geri çağırma (retrieval) adımı tamamlandıktan hemen sonra ve veriler LLM'e (Dil Modeline) iletilmeden hemen önce çalışan bir post-processor'dür. Düğümün orijinal metnini (yani tekil cümleyi), metadata içinde saklanan genişletilmiş bağlamsal pencere metniyle dinamik olarak değiştirir.

3. SentenceTransformerRerank (Yeniden Sıralandırıcı): Arama aşamasında başlangıç benzerlik limiti (similarity_top_k) büyük tutulur (örn: 6). Reranker modeli (örn: BAAI/bge-reranker-base), getirilen bu aday düğümleri sorguyla olan ilişkilerine göre yeniden puanlar ve en alakalı olan ilk N adedi (top_n=2) seçer. Bu işlem, LLM'e gidecek olan bağlaman kalitesini artırırken gürültüyü filtreler.

4. LLAMAINDEX UYGULAMA KODU

LlamaIndex ile bir Cümle Penceresi arama indeksinin ve sorgu motorunun uçtan uca kurulmasını sağlayan Python kodu aşağıda sunulmuştur. Bu kod yapısında gömme modeli (embedding) olarak yerel çalışan bge-small modeli, yeniden sıralandırıcı olarak ise bge-reranker-base modeli tercih edilmiştir:

```
from llama_index import ServiceContext, VectorStoreIndex
from llama_index.node_parser import SentenceWindowNodeParser
from llama_index.indices.postprocessor import MetadataReplacementPostProcessor
from llama_index.indices.postprocessor import SentenceTransformerRerank
from llama_index.llms import OpenAI

# 1. Döğüm Ayırıştırıcı Kurulumu (Cümle Penceresi)
node_parser = SentenceWindowNodeParser.from_defaults(
    window_size=3,
    window_metadata_key="window",
    original_text_metadata_key="original_text"
)

# 2. Servis Bağlamı ve Model Yapılandırması
service_context = ServiceContext.from_defaults(
    llm=OpenAI(model="gpt-3.5-turbo", temperature=0.1),
    embed_model="local:BAAI/bge-small-en-v1.5",
    node_parser=node_parser
)

# 3. Vektör İndeksinin Oluşturulması
index = VectorStoreIndex.from_documents(
    documents, service_context=service_context
)

# 4. Bağlam Değıştirici ve Reranker Bileşenlerinin Tanımlanması
metadata_replacement = MetadataReplacementPostProcessor(
    target_metadata_key="window"
)
rerank = SentenceTransformerRerank(
    model="BAAI/bge-reranker-base", top_n=2
)

# 5. Sorgu Motorunun Oluşturulması (Top_K=6 ve Top_N=2)
query_engine = index.as_query_engine(
    similarity_top_k=6,
    node_postprocessors=[metadata_replacement, rerank]
)
```

5. DENEYSEL PARAMETRE ANALİZİ VE RAG ÜÇLÜSÜ (TRULENS)

Geliştirilen RAG uygulamasının başarısı, TruLens kütüphanesi ve **RAG Üçlüsü (RAG Triad)** metrikleri kullanılarak ölçülmüştür. RAG Üçlüsü, sistemin kalitesini üç boyutta değerlendirir:

- **Bağlam Uygunluğu (Context Relevance):** Geri çağrılan bağlamın soruyla ne kadar doğrudan ilişkili olduğu.
- **Kaynağa Bağlılık (Groundedness):** LLM tarafından üretilen yanıtın her bir cümlesinin geri çağrılan bağlama ne kadar dayandığı (bilgi uydurma/hallucination tespiti).
- **Yanıt Uygunluğu (Answer Relevance):** Üretilen nihai yanıtın sorulan soruya ne kadar doğrudan cevap verdiği.

Deneysel değerlendirmeler kapsamında, cümle penceresi boyutu (Sentence Window Size) parametresi ardışık olarak 1, 3 ve 5 değerleriyle test edilmiş, her bir varyasyonun metrikler, token maliyeti ve sistem kararlılığı üzerindeki etkileri gözlemlenmiştir. Elde edilen deneysel bulgular aşağıdaki tabloda özetlenmiştir:

Pencere Boyutu	Bağlam Uygunluğu (Context Relevance)	Kaynağa Bağlılık (Groundedness)	Yanıt Uygunluğu (Answer Relevance)	Token Maliyeti ve Gecikme
Boyut = 1 (Dar Bağlam)	Çok Düşük (0.57) Çekilen bağlam çok küçük kalır ve gerekli bilgileri kaçırrır.	Düşük / Orta Bilgi yetersizliğinden ötürü LLM kendi ezberini kullanır, puan düşer.	Makul / Orta Soruya cevap verilir ancak kaynak doğruluğu şüphelidir.	Düşük Maliyet ~9,000 token işlenir. Ortalama gecikme 4.57 saniyedir.
Boyut = 3 (Optimal Denge)	Yüksek (0.90) Cümle çevresindeki genişleme sayesinde bağlam kusursuzlaşır.	Mükemmel (1.00) Zengin bağlam sayesinde LLM uydurma yapmaz, kaynağa sadık kalır.	Çok Yüksek Hem doğru hem de tam olarak hedeflenen yanıt üretilir.	Dengeli Maliyet Token tüketimi makul artar, doğruluk/maliyet oranı zirvededir.
Boyut = 5 (Aşırı Bilgi)	Düzleşir / Azalır Gereksiz komşu cümleler bağlama eklenerek gürültü oluşturur.	Düşüş Eğiliminde Aşırı uzun metin LLM'i boğar (overwhelmed) ve ezber bilgiye iter.	Düzleşir / Sabit Boyut 3'e göre bir iyileşme sağlamaz, kararlılık azalır.	Çok Yüksek Maliyet Token kullanımı aşırı artar, gecikme ve maliyet katlanır.

6. KRİTİK PARAMETRE DEĞERLENDİRMELERİ VE BULGULAR

Pencere Boyutu = 1 (Dar Bağlam Sorunu):

Geri çağrılan asıl cümlelerin öncesine ve arkasına sadece birer cümle eklenmesi, LLM'in sorulan soruyu anlamlandırması için çoğunlukla yetersiz kalır. Örneğin, 'ready-fire' ve 'ready-fire-aim' yaklaşımlarının farkını soran karmaşık bir soruda, Pencere Boyutu 1 olduğunda getirilen bağlamın bir kısmı düşük uygunluğa (0.57) sahip olur. Bağlam dar kaldığında, LLM yanıtı oluştururken eksik yerleri kendi eğitim verisinden (ezber bilgi) tamamlamaya zorlanır. Bu durum, LLM'in mantıklı cevaplar üretmesine rağmen yanıtın getirilen bağlamla uyuşmamasına ve dolayısıyla **Kaynağa Bağlılık (Groundedness)** skorunun sıfıra (0) düşmesine sebep olur. Sistem kontrolünü kaybeder.

Pencere Boyutu = 3 (Optimal Denge):

Pencere boyutu 3'e çıkarıldığında, aynı soru için geri çağrılan bağlama komşu 3'er cümle daha eklenir. Bu sayede, aynı metin parçası genişleyerek çok daha bütüncül bir bilgi sunar. Sonuç olarak **Bağlam Uygunluğu (Context Relevance)** skoru 0.57'den 0.90'a yükselir. Genişleyen ve eksiksiz hale gelen bağlam sayesinde, LLM kendi dış bilgisine başvurmak zorunda kalmaz. İhtiyacı olan tüm kanıtları bağlamda bularak yanıtı doğrudan bu verilere dayandırır. Bu sayede **Kaynağa Bağlılık (Groundedness)** skoru kusursuz hale gelerek maksimum seviyeye (1.00) ulaşır. Sistem son derece güvenli ve denetlenebilir çalışır.

Pencere Boyutu = 5 (Aşırı Gürültü ve Maliyet):

Pencere boyutunun 5'e yükseltilmesi, her sorguda dil modeline çok daha fazla cümlelerin (token) iletilmesine yol açar. Bu durum operasyonel maliyeti ve gecikmeyi doğrusal olarak artırır. Ancak Bağlam Uygunluğu ve Yanıt Uygunluğu metriklerinde herhangi bir artış sağlamaz (grafik düzleşir). Daha da önemlisi, bağlam boyutu aşırı büyük olduğunda LLM sentezleme adımında gereksiz bilgilerle boğulur (overwhelmed). Metin yığınının içinde kaybolan model, odaklanması gereken detayları kaçırarak yine eğitim aşamasındaki ezber bilgilerine kayma eğilimi gösterir. Bu durum, **Kaynağa Bağlılık (Groundedness)** skorunun Boyut 3 seviyesinden daha aşağıya düşmesine sebep olur. Dolayısıyla, büyük bağlam pencereleri her zaman daha iyi sonuç üretmez ve sistem kararlılığını bozar.

7. EN İYİ UYGULAMA ÖNERİLERİ

- **Optimal Pencere Seçimi:** Sistem tasarımlarında başlangıç parametresi olarak her zaman **window_size=3** tercih edilmelidir. Bu parametre, hem finansal maliyeti (token tüketimi) kontrol altında tutar hem de RAG Üçlüsü metriklerini en üst düzeye ulaştıran kanıtlanmış bir denge noktasıdır.
- **Reranker Entegrasyonu:** Başlangıç geri çağırma adımı (similarity_top_k) her zaman geniş tutulmalıdır (örn. 6). Geniş bağlamın getirdiği gürültüyü temizlemek ve maliyeti düşürmek amacıyla araya mutlaka bge-reranker gibi bir **yeniden sıralandırma (reranker)** katmanı eklenerek en alakalı top_n=2 düğüm LLM'e iletilmelidir.
- **Dinamik Değerlendirme Döngüsü:** Sistem üretime alınmadan önce TruLens benzeri araçlarla 'RAG Triad' değerlendirmesi otomatikleştirilmeli; özellikle Groundedness skoru takip edilerek modelin uydurma yapıp yapmadığı veya ezber bilgisine kayıp kaymadığı sürekli izlenmelidir.